

# KẾ HOẠCH NÂNG CẤP VÀ KIỂM SOÁT XUNG ĐỘT HỆ THỐNG T2M-APP

Tài liệu hướng dẫn tổng hợp toàn bộ lộ trình phát triển nâng cao, quy tắc bảo vệ mã nguồn cũ và quy trình thực hiện tin gọn (Vibe Coding) dành cho Người quản lý đội xe (5 xe đầu kéo, 12 rơ-moóc).

## 1. BỐN QUY TẮC BẤT BIẾN BẢO VỆ HỆ THỐNG (INVARIANTS)

Trước khi đưa bất kỳ chỉ thị nào cho AI viết mã nguồn, thợ mã (Vibe Coder) phải bắt buộc ép AI tuân thủ nghiêm ngặt 4 quy tắc nền tảng sau để không phá vỡ cấu trúc đang chạy ổn định:

| Hạng mục cốt lõi        | Nguyên tắc kiểm soát mã nguồn chống xung đột                                                                                                                                                                              |
|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Offline-First Engine | Giữ nguyên vẹn cơ chế hoạt động của hook useSyncEngine.ts và hệ thống lưu nháp ngầm IndexedDB (idb-keyval). Tuyệt đối không gọi trực tiếp API Supabase tại các form nhập liệu để đảm bảo mất mạng vẫn lưu được phiếu.     |
| 2. Vòng Đòi Lốp Xe      | Logic đảo lốp hoặc luân chuyển bánh xe bắt buộc chỉ dùng câu lệnh UPDATE trường vi_tri_lap trên bảng quan_ly_vo_xe. NGHIÊM CẤM sử dụng lệnh INSERT lốp mới để tránh trùng lặp tài sản và lỗi đồng bộ khóa ngoại.          |
| 3. Tính Toán Chi Phí    | Tuyệt đối không thực hiện các phép tính cộng dồn tổng tiền (tong_chi_phi, tong_vat_tu) ở phía giao diện người dùng (Frontend). Hệ thống kế thừa và sử dụng hoàn toàn Server-side PostgreSQL Trigger để tính toán tự động. |
| 4. Định Vị Chống Gian   | Mọi module liên quan đến kiểm tra sai lệch tọa độ mở phiếu của                                                                                                                                                            |

| Hạng mục cốt lõi | Nguyên tắc kiểm soát mã nguồn chống xung đột                                                                                                  |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| Lặn              | thợ máy bắt buộc phải gọi lại hàm tính khoảng cách toán học Haversine sẵn có tại file src/lib/haversine.ts, không cho AI tự viết lại hàm mới. |

## 2. CHI TIẾT 5 GIAI ĐOẠN PHÁT TRIỂN NÂNG CAO (5 PHASES)

### Phase 1: Sơ đồ vỏ xe trực quan & Quản lý vòng đời lốp

- **Task 1.1 (UI Component):** Xây dựng file độc lập src/features/master-data/components/TireVisualMap.tsx hiển thị sơ đồ lưới (Grid) cấu trúc bánh xe. Đầu kéo hiển thị 6 hoặc 10 bánh; rơ-moóc hiển thị 8 hoặc 12 bánh tùy cấu hình trực. Kèm trạng thái ô trống nét đứt nếu chưa gắn lốp.
- **Task 1.2 (Interactive Dialog):** Click vào vị trí bánh xe mở Modal Shadcn UI hiển thị: Số Serial lốp (id\_vo), Thương hiệu, Độ sâu gai (mm), nút xem ảnh Serial và ảnh đo gai thực tế (Kế thừa trường dữ liệu mới từ cấu trúc fix\_tire\_management.sql).
- **Task 1.3 (Rotation Action):** Viết handler chọn 2 vị trí bánh xe bất kỳ, thực hiện hoán đổi vị trí bằng câu lệnh UPDATE duy nhất lên Supabase, đồng thời đẩy bản ghi sự kiện điều chuyển vào bảng tire\_history để theo dõi vết.

### Phase 2: Trung tâm cảnh báo tự động (Dual-Trigger Alerts Engine)

- **Task 2.1 (Database View):** Viết mã SQL tạo View view\_active\_alerts trên Supabase quét tự động: Cảnh báo bảo dưỡng đầu kéo (KM chạy thêm kể từ lần bảo dưỡng gần nhất  $\geq 10,000$ km) và Cảnh báo đăng kiểm định kỳ (Số ngày còn lại tới hạn đăng kiểm  $\leq 15$  ngày).
- **Task 2.2 (Alerts Dashboard UI):** Tạo trang quản trị trung tâm src/app/admin/alerts/page.tsx hiển thị danh sách cảnh báo phân loại theo màu sắc (Đỏ: Quá hạn khẩn cấp, Vàng: Sắp đến hạn).
- **Task 2.3 (GPS Fraud Feed):** Thiết kế khu vực riêng hiển thị các phiếu bảo trì có cờ canh\_bao\_gps = true. Ghi rõ tên thợ máy, mã phiếu, khoảng cách lệch tâm bãi bốc xếp thực tế và tích hợp link bản đồ mini định vị tọa độ gian lận.

### Phase 3: Định danh xe bằng mã QR (QR Code Integration)

- **Task 3.1 (QR Generator):** Tích hợp thư viện qrcode.react vào bảng danh mục xe giúp Admin tự động xuất mã QR tĩnh định dạng t2m://vehicle/[id] để in dán trực tiếp lên thành xe/móc tại bãi.
- **Task 3.2 (QR Scanner Mobile):** Tạo hook useQRScanner.ts tích hợp html5-qrcode gọi camera quét mã trực tiếp trên trình duyệt điện thoại của thợ máy.
- **Task 3.3 (Smart Routing & Autofill):** Khi thợ máy quét thành công mã QR xe, ứng dụng tự động điều hướng vào TicketForm, điền sẵn chính xác ID xe/Biển số và khóa cứng trường này để chống chọn nhầm xe.

## Phase 4: Micro-animations & Hoàn thiện giao diện phụ

- **Task 4.1 (Tab Transition):** Sử dụng framer-motion bọc ngoài cấu phần chuyển đổi tab danh mục (Xe, Vật tư, Khách hàng, Nhà cung cấp) tại master-data tạo hiệu ứng mượt mà (Fade-in/Slide-up).
- **Task 4.2 (CRUD Dialog Completion):** Hoàn thiện 100% biểu mẫu thêm/sửa/xóa cho tab Khách hàng và Nhà cung cấp (đã được bóc tách hook từ bản cập nhật ngày 17/05).
- **Task 4.3 (Global Poka-yoke Audit):** Áp dụng sự kiện onFocus={e}=> e.target.select()} cho toàn bộ các ô dữ liệu số còn lại trong các biểu mẫu để hỗ trợ thợ máy dễ nhập đề số mới khi tay dính dầu mỡ.

## Phase 5: Dự báo ngân sách bảo trì & Báo cáo giám đốc chuyên sâu

- **Task 5.1 (Predictive Algorithm):** Viết hàm tính toán dựa trên tần suất số KM tăng trưởng trung bình của đầu kéo và chu kỳ hao mòn của vật tư tiêu hao (nhớt, lốp, lọc) để dự phòng các xe chạm mốc sửa chữa trong 30 ngày tới.
- **Task 5.2 (Forecast UI Component):** Thêm widget đồ họa "Ngân sách dự trù tháng tới" hiển thị số tiền ước tính và danh sách vật tư cần mua lên AnalyticsDashboard.tsx.
- **Task 5.3 (Super Excel Export Upgrade):** Nâng cấp hàm xuất Excel hiện tại từ cấu trúc 3-sheets lên cấu trúc 4-sheets, bổ sung sheet thứ 4 mang tên "Du\_Bao\_Ngan\_Sach" phục vụ nộp trực tiếp cho Giám đốc duyệt chi dòng tiền sớm.

## 3. QUY TRÌNH HÀNH ĐỘNG VÀ MẪU PROMPT TIẾT KIỆM ĐẾN 90% TOKEN

Để quá trình Vibe Coding diễn ra nhanh chóng, ít tốn chi phí và không làm AI bị loãng ngữ cảnh, hãy áp dụng quy trình 2 bước sau:

1. **Bước 1 (Cô lập UI):** Luôn yêu cầu AI viết file giao diện tĩnh (Mock Data) trước, đặt độc lập trong thư mục components của feature, chưa cho kết nối database vội.

2. **Bước 2 (Gắn kết Logic):** Khi UI đã chuẩn, copy đoạn Prompt nguyên tử nén gọn bên dưới kèm theo yêu cầu nghiệp vụ chi tiết của task để AI thực hiện kết nối dữ liệu.

**MẪU PROMPT NÉN SIÊU TIẾT KIỆM TOKEN:**

Cấu trúc dự án: Next.js 14, Zustand, React Query, Supabase.

Tên bảng chuẩn: vehicles, skus, garages, chi\_tiet\_phieu\_bao\_tri.

4 Luật bất biến: (1) Giữ cơ chế offline của useSyncEngine.ts. (2) Đảm bảo chỉ dùng UPDATE bảng 'quan\_ly\_vo\_xe', CẤM lệnh INSERT lớp trùng. (3) Chi phí do Server trigger tính, cấm tính ở frontend. (4) Định vị dùng hàm haversine.ts có sẵn.

Yêu cầu hành động: [Dán nội dung Task chi tiết cần thực hiện vào đây]